신원 관리 및 데이터 무결성 시스템

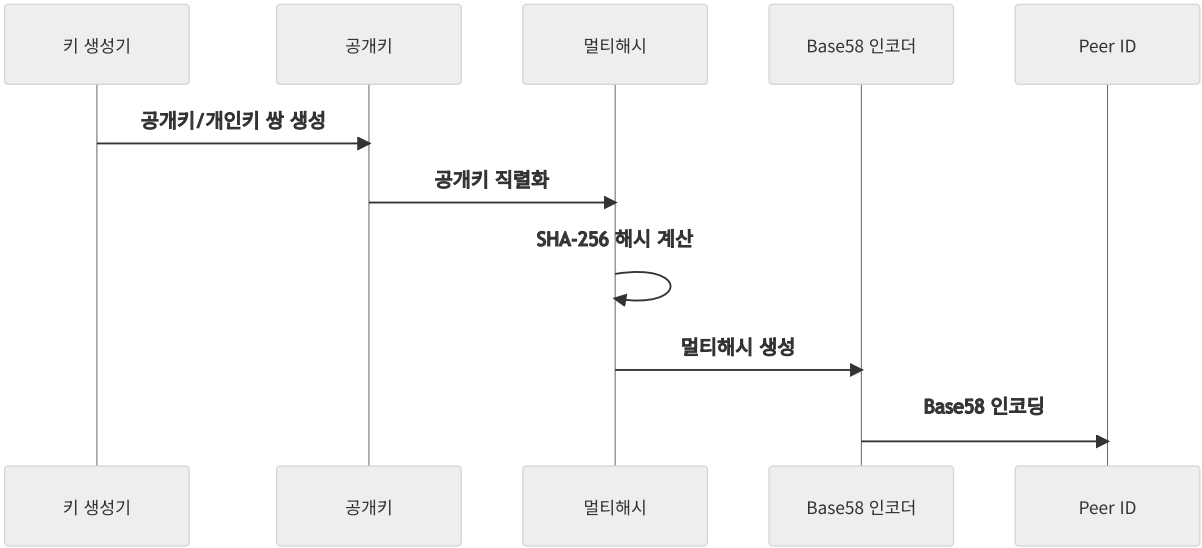
개요

IPFS의 신원 관리 시스템은 분산 네트워크에서 피어의 신원을 확인하고 데이터의 무결성을 보장하는 핵심 메커니즘입니다. 이 시스템은 암호학적 해시와 디지털 서명을 기반으로 하여 중앙 인증 기관 없이도 안전한 피어 간 통신을 가능하게 합니다.

Peer ID 시스템

Peer ID 생성 과정

Peer ID는 IPFS 네트워크에서 각 노드를 고유하게 식별하는 암호학적 식별자입니다:



1단계: 키 쌍 생성

```go
// config/init.go에서 키 생성
priv, pub, err := crypto.GenerateKeyPair(crypto.RSA, 2048)
// 또는
priv, pub, err := crypto.GenerateEd25519Key(rand.Reader)
```

2단계: 공개키 직렬화

```go
// 공개키를 바이트 배열로 직렬화
pubKeyBytes, err := crypto.MarshalPublicKey(pub)
if err != nil {
return err
}
```

3단계: 해시 계산 및 멀티해시 생성

```go
// SHA-256 해시 계산
hasher := sha256.New()
hasher.Write(pubKeyBytes)
hash := hasher.Sum(nil)

// 멀티해시 생성 (해시 함수 정보 포함)
multihash, err := multihash.Encode(hash, multihash.SHA2_256)
if err != nil {
```

```
return err
}
```

4단계: Peer ID 생성

```
// libp2p의 peer.IDFromPublicKey 함수 사용
peerID, err := peer.IDFromPublicKey(pub)
if err != nil {
return err
}

// 결과: "12D3KooWGRPLCW..." 형태의 Base58 인코딩된 문자열
```

Peer ID 구조 분석

멀티해시 구성

```
Peer ID = Base58(Multihash(PublicKey))

Multihash 구조:
<hash-function-code><hash-length><hash-digest>

예시:
0x12 (SHA-256) + 0x20 (32바이트) + [32바이트 해시]
```

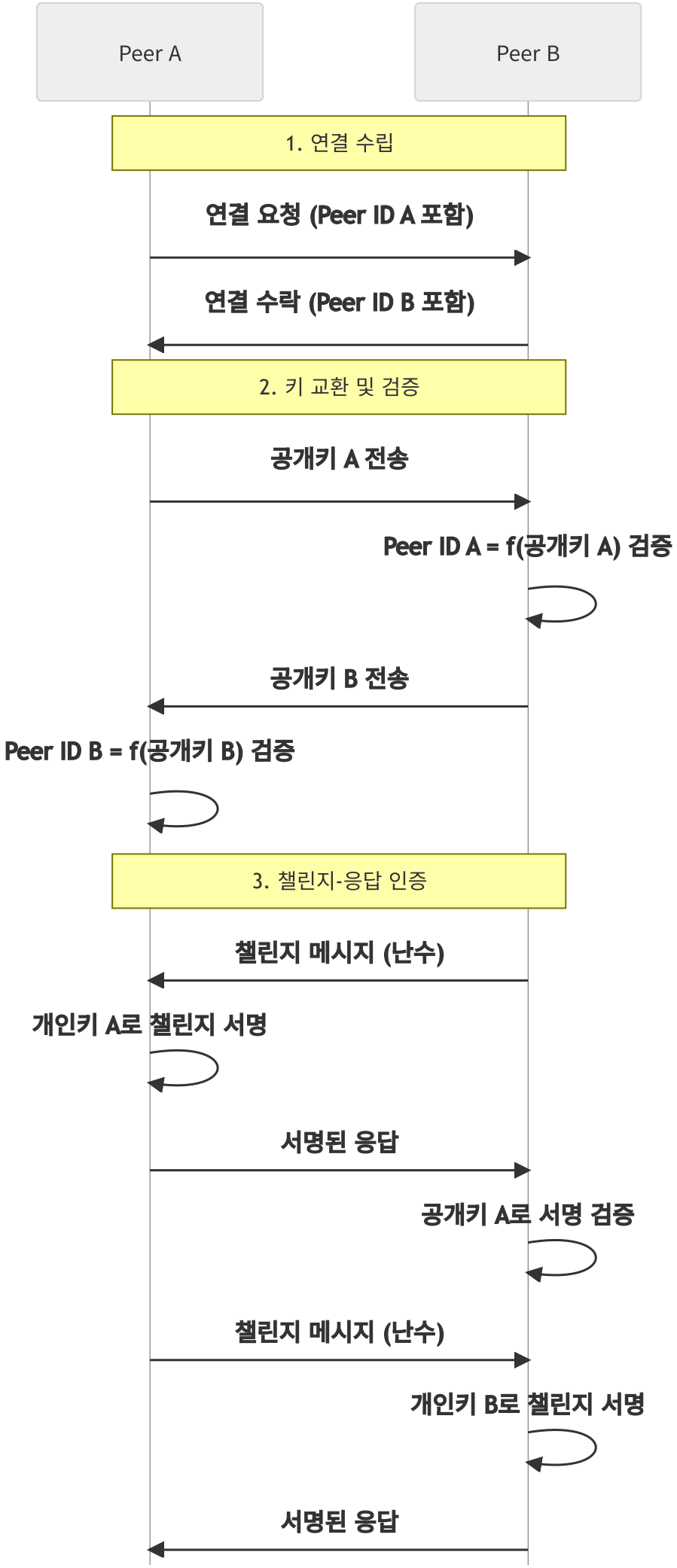
검증 가능한 특성

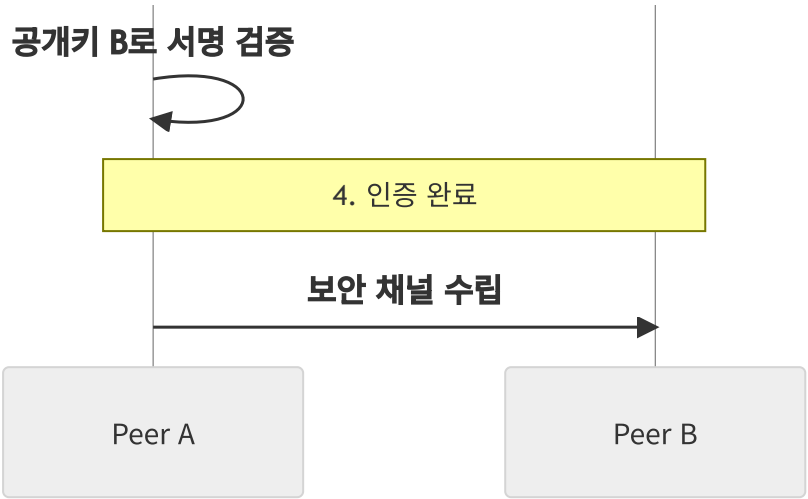
- **결정적**: 같은 공개키는 항상 같은 Peer ID 생성
- **고유성**: 서로 다른 키는 서로 다른 Peer ID 생성 (충돌 확률 극히 낮음)
- **검증 가능**: 공개키로부터 Peer ID 재생성하여 검증 가능

피어 인증 과정

연결 수립 시 인증

IPFS 피어 간 연결 수립 시 다음과 같은 인증 과정을 거칩니다:


Peer A
Peer B
1. 연결 수립
연결 요청 (Peer ID A 포함)
연결 수락 (Peer ID B 포함)
2. 키 교환 및 검증
공개키 A 전송
Peer ID A = f(공개키 A) 검증
공개키 B 전송
Peer ID B = f(공개키 B) 검증
3. 챌린지-응답 인증
챌린지 메시지 (난수)
개인키 A로 챌린지 서명
서명된 응답
공개키 A로 서명 검증
챌린지 메시지 (난수)
개인키 B로 챌린지 서명
서명된 응답



인증 구현 세부사항

1. Peer ID 검증

```
// core/node/identity.go의 검증 로직
func PrivateKey(sk crypto.PrivKey) func(id peer.ID) (crypto.PrivKey, error) {
return func(id peer.ID) (crypto.PrivKey, error) {
// 개인키로부터 Peer ID 재생성
id2, err := peer.IDFromPrivateKey(sk)
if err != nil {
return nil, err
}

// 설정된 Peer ID와 일치하는지 확인
if id2 != id {
return nil, fmt.Errorf("private key in config does not match id: %s != %s", id,
}
return sk, nil
```

```
}

}
```

2. 디지털 서명 생성 및 검증

```
// 메시지 서명 생성
func SignMessage(privateKey crypto.PrivKey, message []byte) ([]byte, error) {
signature, err := privateKey.Sign(message)
if err != nil {
return nil, err
}
return signature, nil
}

// 서명 검증
func VerifySignature(publicKey crypto.PubKey, message []byte, signature []byte)
valid, err := publicKey.Verify(message, signature)
if err != nil {
return err
}
if !valid {
return errors.New("signature verification failed")
}
return nil
```

```
}
```

데이터 무결성 보장 메커니즘

해시 기반 콘텐츠 검증

IPFS는 콘텐츠 주소 지정(Content Addressing)을 통해 데이터 무결성을 보장합니다:

CID 기반 검증 과정

```
func VerifyBlockIntegrity(expectedCID cid.Cid, blockData []byte) error {
// 1. 블록 데이터로부터 해시 계산
hasher, err := multihash.GetHasher(expectedCID.Hash().Code())
if err != nil {
return fmt.Errorf("unsupported hash function: %v", err)
}
hasher.Write(blockData)
calculatedHash := hasher.Sum(nil)
// 2. 계산된 해시와 CID의 해시 비교
expectedHash := expectedCID.Hash().Digest()
if !bytes.Equal(calculatedHash, expectedHash) {
return fmt.Errorf("block integrity check failed: hash mismatch")
}
return nil
```

```
}
```

무결성 검증 단계

1. **데이터 수신**: 네트워크에서 블록 데이터 수신
2. **해시 재계산**: 수신된 데이터의 해시 재계산
3. **CID 비교**: 계산된 해시와 요청된 CID의 해시 비교
4. **검증 결과**: 일치 시 유효, 불일치 시 오류 반환

서명 기반 데이터 인증

IPNS 레코드 서명

```
// IPNS 레코드 생성 및 서명
func CreateIPNSRecord(privateKey crypto.PrivKey, value path.Path, validity time
// 레코드 데이터 구성
entry := &pb.IpnsEntry{
Value: []byte(value.String()),
ValidityType: pb.IpnsEntry_EOL,
Validity: []byte(time.Now().Add(validity).Format(time.RFC3339)),
Sequence: getCurrentSequence(),
Ttl: uint64(validity.Nanoseconds()),
}
// 서명할 데이터 준비
dataForSig, err := ipnsEntryDataForSig(entry)
if err != nil {
```

```
return nil, err
}
// 개인키로 서명
signature, err := privateKey.Sign(dataForSig)
if err != nil {
return nil, err
}
entry.Signature = signature
// 공개키 포함
pubKeyBytes, err := crypto.MarshalPublicKey(privateKey.GetPublic())
if err != nil {
return nil, err
}
entry.PubKey = pubKeyBytes
return entry, nil
}
```

IPNS 레코드 검증

```
func VerifyIPNSRecord(entry *pb.IpnsEntry) error {
// 공개키 추출
pubKey, err := crypto.UnmarshalPublicKey(entry.PubKey)
if err != nil {
```

```
return err
}
// Peer ID 검증
expectedPeerID, err := peer.IDFromPublicKey(pubKey)
if err != nil {
return err
}
// 서명 검증을 위한 데이터 준비
dataForSig, err := ipnsEntryDataForSig(entry)
if err != nil {
return err
}
// 서명 검증
valid, err := pubKey.Verify(dataForSig, entry.Signature)
if err != nil {
return err
}
if !valid {
return errors.New("IPNS record signature verification failed")
}
// 유효성 검사
if err := validateIPNSRecord(entry); err != nil {
return err
```

```
}
return nil
}
```

신뢰 모델 및 보안 정책

분산 신뢰 모델

IPFS는 중앙 인증 기관 없이 작동하는 분산 신뢰 모델을 사용합니다:

자체 서명 인증서 (Self-Signed Certificates)

- **PKI 불필요**: 중앙 인증 기관이나 인증서 체인 불필요
- **암호학적 신원**: 공개키 암호화를 통한 신원 증명
- **검증 가능성**: 누구나 공개키로 신원 검증 가능

신뢰 관계 구축

```
// 피어 신뢰도 관리 예시
type PeerTrustManager struct {
trustedPeers map[peer.ID]TrustLevel
blacklistedPeers map[peer.ID]time.Time
mutex sync.RWMutex
}

type TrustLevel int
```

```
const (
Untrusted TrustLevel = iota
LowTrust
MediumTrust
HighTrust
FullyTrusted
)

func (ptm *PeerTrustManager) UpdateTrust(peerID peer.ID, behavior PeerBehavior)
ptm.mutex.Lock()
defer ptm.mutex.Unlock()
currentTrust := ptm.trustedPeers[peerID]
switch behavior {
case GoodBehavior:
if currentTrust < FullyTrusted {
ptm.trustedPeers[peerID] = currentTrust + 1
}
case BadBehavior:
if currentTrust > Untrusted {
ptm.trustedPeers[peerID] = currentTrust - 1
} else {
ptm.blacklistedPeers[peerID] = time.Now()
```

```
}

}

}
```

보안 정책 구현

1. 키 검증 정책

```
// 허용된 키 타입 및 크기 검증
func ValidateKeyPolicy(pubKey crypto.PubKey) error {
switch pubKey.Type() {
case crypto.RSA:
// RSA 키는 최소 2048비트 요구
if pubKey.(*rsa.PublicKey).Size() < 256 { // 2048비트 = 256바이트
return errors.New("RSA key too small, minimum 2048 bits required")
}
case crypto.Ed25519:
// Ed25519는 항상 256비트로 고정
return nil
case crypto.Secp256k1:
// Secp256k1 허용
return nil
default:
return errors.New("unsupported key type")
```

```
}

return nil

}
```

2. 해시 함수 허용 목록

```
// 안전한 해시 함수만 허용

var AllowedHashFunctions = map[uint64]bool{

multihash.SHA2_256: true,

multihash.SHA2_512: true,

multihash.SHA3_256: true,

multihash.SHA3_512: true,

multihash.BLAKE2B_MIN + 31: true, // Blake2b-256

multihash.BLAKE2S_MIN + 31: true, // Blake2s-256

}


func ValidateHashFunction(code uint64) error {

if !AllowedHashFunctions[code] {

return fmt.Errorf("hash function %d not allowed", code)

}

return nil

}
```

공격 방어 메커니즘

1. Sybil 공격 방어

- **리소스 기반 제한**: 키 생성 비용으로 대량 신원 생성 억제
- **평판 시스템**: 피어 행동 기반 신뢰도 관리
- **네트워크 분석**: 비정상적인 연결 패턴 감지

2. Eclipse 공격 방어

- **다양한 피어 연결**: 여러 네트워크 경로를 통한 피어 발견
- **부트스트랩 노드**: 신뢰할 수 있는 초기 연결점 제공
- **DHT 다양성**: 분산 해시 테이블을 통한 라우팅 다양성

3. 중간자 공격 방어

- **종단간 암호화**: libp2p의 TLS/Noise 프로토콜 사용
- **키 지문 검증**: 연결 시 피어 키 지문 확인
- **인증서 고정**: 알려진 피어의 키 고정

성능 최적화

1. 서명 검증 최적화

```
// 서명 검증 결과 캐싱
type SignatureCache struct {
cache  map[string]bool
mutex sync.RWMutex
```

```
ttl time.Duration
}

func (sc *SignatureCache) VerifyWithCache(pubKey crypto.PubKey, message, signatu
// 캐시 키 생성
cacheKey := fmt.Sprintf("%x:%x:%x", pubKey.Raw(), message, signature)
sc.mutex.RLock()
if result, exists := sc.cache[cacheKey]; exists {
sc.mutex.RUnlock()
return result, nil
}
sc.mutex.RUnlock()
// 실제 검증 수행
valid, err := pubKey.Verify(message, signature)
if err != nil {
return false, err
}
// 결과 캐싱
sc.mutex.Lock()
sc.cache[cacheKey] = valid
sc.mutex.Unlock()
return valid, nil
```

```
}
```

2. 해시 계산 최적화

- **하드웨어 가속**: SHA-NI 등 CPU 확장 기능 활용
- **병렬 처리**: 대용량 데이터의 청크 단위 병렬 해싱
- **스트리밍 해시**: 메모리 효율적인 스트리밍 해시 계산

모니터링 및 감사

보안 이벤트 로깅

```
type SecurityEvent struct {
Timestamp time.Time
EventType  string
PeerID peer.ID
Description  string
Severity SecurityLevel
}

func  LogSecurityEvent(event SecurityEvent) {
log.WithFields(logrus.Fields{
"timestamp": event.Timestamp,
"event_type": event.EventType,
```

```
"peer_id": event.PeerID.String(),

"description": event.Description,

"severity": event.Severity,

}).Warn("Security event detected")

}
```

정기 보안 감사

- **키 무결성 검사**: 저장된 키의 무결성 정기 확인
- **피어 신뢰도 분석**: 비정상적인 피어 행동 패턴 분석
- **네트워크 연결 감사**: 의심스러운 연결 패턴 모니터링

이러한 종합적인 신원 관리 및 데이터 무결성 시스템을 통해 IPFS는 분산 환경에서도 높은 수준의 보안을 제공합니다.

참고 자료

- 암호화 시스템 및 키 관리
- 콘텐츠 차단 및 접근 제어
- 콘텐츠 주소 지정 시스템
- [libp2p 네트워킹 보안](.../02-networking/libp2p-integration